开发者社区

DTK

DTK 26.10
HIP 最佳实践手册

DTK-261000-014-0915-929

目　录

1 DCU 程序基础优化方法 1
1.1 GEMM 及其分块策略 1
1.2 基于 DCU 的初步实现 5
1.3 使用寄存器实现数据复用 8
1.4 数据预取隐藏延迟 12
1.5 使用 LDS 进行访存优化 14
1.6 减少 bank 冲突 21
2 并行规约问题 25
3 全局内存合并访问 35
4 总结 41

开发者社区

1 DCU 程序基础优化方法

我们以 GEMM 程序为例，通过逐步对程序进行优化，阐述 DCU 编程中的一些常用的优化策略。

1.1 GEMM 及其分块策略

什么是 GEMM？它的英文全称是 GEneral Matrix Multiplication。GEMM 运算广泛的应用于科学计算领域，比如：航空航天、天气预报、石油勘探、流体力学等。近些年来，随着深度学习的兴起以及深度学习模型和 GEMM 运算的高度相关性，GEMM 运算及其优化变得更加重要。

假设 A:M 行 K 列，B:K 行 N 列，C:M 行 N 列，则计算的伪代码如下：

```
for(int m = 0; m < M; m++)
{
    for(int n = 0; n < N; n++)
    {
        for(int k = 0; k < K; k++)
        {
            C[m][n] += A[m][k]*B[k][n];
        }
    }
}
```

该伪代码计算操作总数为 2MNK 次（其中，M,N,K 分别表示三层循环执行的次数，2 表示循环最内层的一次乘累加操作），内存访问操作总数为 4MNK（其中，4 表示对 A 矩阵，B 矩阵以及 C 矩阵的内存访问，需要先读取内存，乘累加完成后存储，忽略对初始化时的操作）。如果计算 C 矩阵标记出元素的右边相邻的一个元素，则 A 矩阵相同行的元素会被重复读取一次；同样地，如果计算 C 矩阵标记处元素的下边相邻的一个元素，则 B 矩阵相同列的元素会被重复读取一次。由此可见，GEMM 计算中存在大量的访存操作并且数据的重用性很低。

那么基于 GEMM 算法的这个伪代码，如何提高数据的重用性并减少访存，就是一个重要的优化策略。目前，在很多的 GEMM 优化方法中，会使用如下一

种循环的方式进行计算：

```
for(int k = 0; k < K; k++)
{
    for(int m = 0; m < M; m++)
    {
        for(int n = 0; n < N; n++)
        {
            C[m][n] += A[m][k]*B[k][n];
        }
    }
}
```

在这种计算方法中，最外层循环每变化一次，会读取 A 矩阵的一列和 B 矩阵的一行，其中所有的元素两两相乘，得到 C 矩阵的一“层”，C 矩阵的每个位置都会被写入一次。此时，这一行、一列元素就完成了自己的计算任务，就可以被扔掉了，计算如图 1.1 所示。

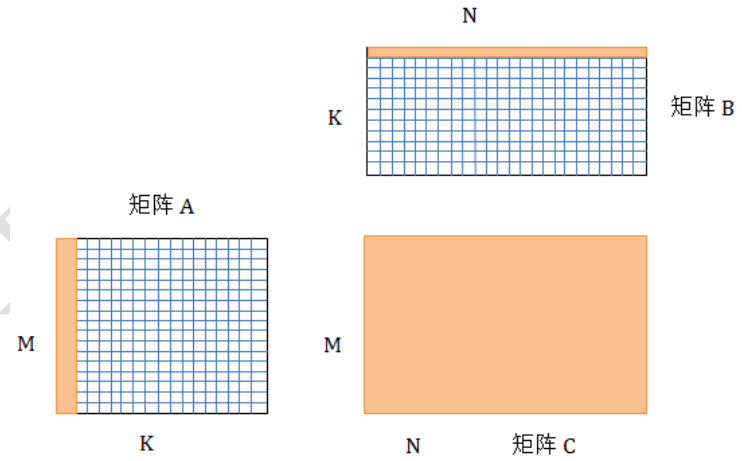


图 1.1 C 矩阵每“层”计算方式

假设矩阵的维度尺寸很小，我们按照此种方式，可以每次将 A 矩阵的一列和 B 矩阵的一行，读取到寄存器中，然后从寄存器中读取数据计算中 C 矩阵每个位置的部分值，避免每次从 A 矩阵、B 矩阵对应的内存中，不断的访存，实现数据的复用，减少 GEMM 算法访存的开销。

如果矩阵的维度尺寸比较大，我们没有办法一次将 A 矩阵的一列和 B 矩阵

的一行，读取到寄存器中，这个时候该如何处理呢？这就要引出 GEMM 优化中的分块了，我们可以在每个分块后的小矩阵中使用这种计算方式。分块策略包含：

- 以 C 矩阵为对象沿 m、n 方向的分块；
- 以 A 矩阵、B 矩阵为对象沿 k 方向的分块，分块方式如图 1.2 所示。

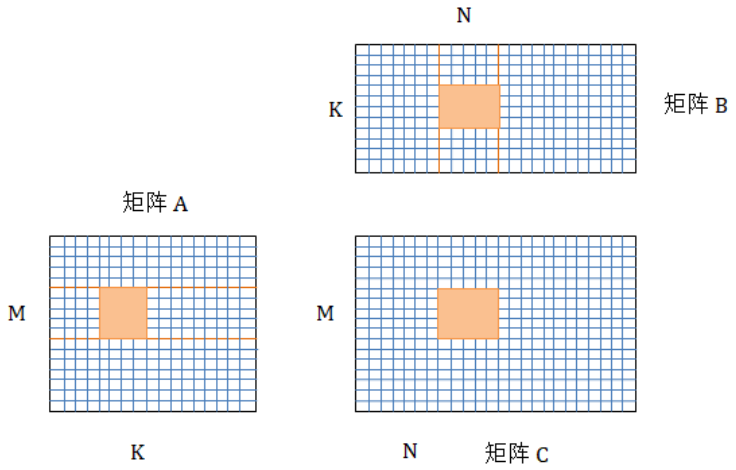


图 1.2 GEMM 计算分块方法

以 4x4 的分块与 K 维度 depth 为 4 的方式分块后，计算伪代码如下：

```
for(int m = 0; m < M; m+=4)
{
    for(int n = 0; n < N; n+=4)
    {
        C[m+0][n+0...3] = 0;
        C[m+1][n+0...3] = 0;
        C[m+2][n+0...3] = 0;
        C[m+3][n+0...3] = 0;
        for(int k = 0; k < K; k+=4)
        {
            for(int subk = k; subk < k+4; subk++){
                for(int subm = m; subm < m+4; subm++){
                    for(int subn = n; subn< n+4; subn++){
                        C[subm][subn] += A[subm][subk]*B[subk][subn];
                    }
```

```
                }
            }
        }
    }
}
```

对分块后，子块计算的伪代码进行循环展开，可得：

```
for(int m = 0; m < M; m+=4)
{
    for(int n = 0; n < N; n+=4)
    {
        C[m+0][n+0...3] = 0;
        C[m+1][n+0...3] = 0;
        C[m+2][n+0...3] = 0;
        C[m+3][n+0...3] = 0;
        for(int k = 0; k < K; k+=4)
        {
            C[m+0...3][n+0...3] += A[m+0...3][k+0] * B[k+0][n+0...3];
            C[m+0...3][n+0...3] += A[m+0...3][k+1] * B[k+1][n+0...3];
            C[m+0...3][n+0...3] += A[m+0...3][k+2] * B[k+2][n+0...3];
            C[m+0...3][n+0...3] += A[m+0...3][k+3] * B[k+3][n+0...3];
        }
    }
}
```

上图中将输出的计算矩阵 C 拆分为 4x4 的小块，计算该块的输出时，将 K 维度也进行了分块。在对 M 和 N 展开时，我们可以分别复用 A 矩阵 和 B 矩阵的数据（A 矩阵、B 矩阵的每个元素均可以复用 4 次）；在对 K 展开时，我们可以将部分和累加在寄存器中，最内层循环一次迭代结束时一次写到 C 的内存中。那么内存访问的次数变为：$2MN + 1/4MNK + 1/4MNK \approx 1/2MNK$，相对于最原始版本有 8 倍的改进。

本小节以一个小例子，对 GEMM 及其分块策略进行介绍。下面我们分不同的小节，以 M=64，N=64，K=128，TN 类型的 GEMM 为例，编写代码并采用不同的优化策略逐步进行优化。

1.2 基于 DCU 的初步实现

在我们的初步代码实现中，以 C 矩阵为对象，将其以 4x4 的小矩阵进行划分，如下：

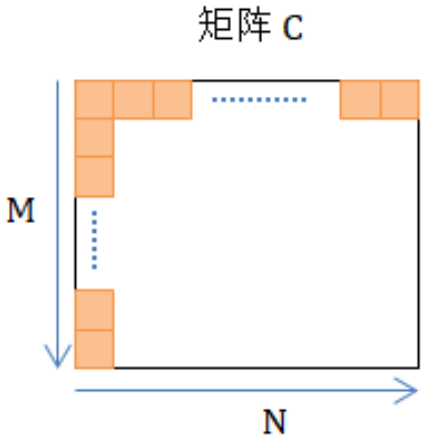


图 1.3 C 矩阵以 4x4 的小矩阵分块

由前面的 M=64，N=64 的条件，我们可以将 C 矩阵分为 16x16 个子块。在本小节，基于 DCU 的实现中，我们用每个线程负责一个 4x4 小块区域的计算。实现 kernel 代码如下：

```
__global__ void three_cycle(double *src_a, double *src_b, double *dst_c, double alpha,
double beta, int size_m, int size_n, int size_k)
{
    unsigned int serial    = threadIdx.x;
    unsigned int index_j = (serial >> 4)*4;
    unsigned int index_i = (serial & 15)*4;

    unsigned int offset_c = index_i * size_n + index_j;
    unsigned int offset_a = index_i;
    unsigned int offset_b = index_j;
```

```
    double *tmp_c = dst_c + offset_c;
    double *tmp_a = src_a + offset_a;
    double *tmp_b = src_b + offset_b;

    int i,j,k;
    double a0,a1,a2,a3,b0,b1,b2,b3,c0;
    for(i=0; i<4; i++)
    {
        for(j=0; j<4; j++)
        {
            double sum = 0;
            for(k=0; k<size_k; k+=4)
            {
                a0 = *(tmp_a + k*size_m + i);
a1 = *(tmp_a + (k + 1)*size_m + i);
                a2 = *(tmp_a + (k + 2)*size_m + i);
a3 = *(tmp_a + (k + 3)*size_m + i);
                b0 = *(tmp_b + k*size_n + j);
b1 = *(tmp_b + (k + 1)*size_n + j);
                b2 = *(tmp_b + (k + 2)*size_n +j);
b3 = *(tmp_b + (k + 3)*size_n + j);
            sum += a0 * b0 + a1 * b1 + a2 * b2 + a3 * b3;
            }
            c0 = *(tmp_c+j);
            sum = sum * alpha + beta * c0;
            *(tmp_c+j) = sum;
        }
        tmp_c = tmp_c + size_n;
    }
```

```
}
```

在该代码中，我们使用 1 个线程块，并且该线程块只有 x 维度不为 1，线程块中有 256 个线程。每个线程，映射到对应的 4x4 的小矩阵块：

```
unsigned int serial    = threadIdx.x;
unsigned int index_j = (serial >> 4)*4;
unsigned int index_i = (serial & 15)*4;
```

为了便于理解，当前线程的待处理的小矩阵块的左上角元素所在的 C 矩阵位置如下图所示。

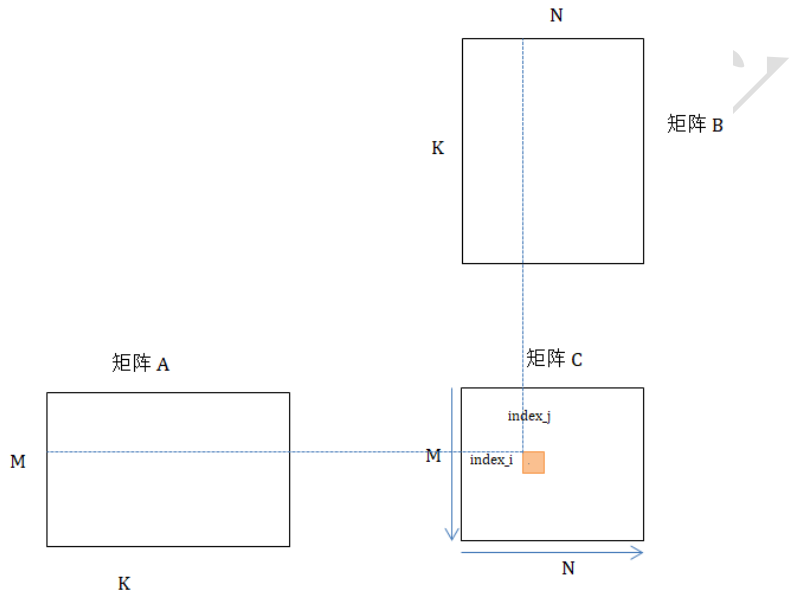


图 1.4 当前线程处理的小矩阵块左上角的索引

其中，(serial >> 4)=serial/16，serial & 15 = serial%16。

接下来，计算出该小矩阵块区域 GEMM 求解，所需要的的 A 矩阵、B 矩阵、C 矩阵的起始偏移，代码如下：

```
unsigned int offset_c = index_i * size_n + index_j;
unsigned int offset_a = index_i;
unsigned int offset_b = index_j;
double *tmp_c = dst_c + offset_c;
double *tmp_a = src_a + offset_a;
double *tmp_b = src_b + offset_b;
```

当前线程处理的小矩阵块所需的矩阵 A、矩阵 B 的区域如图 1.5 所示。

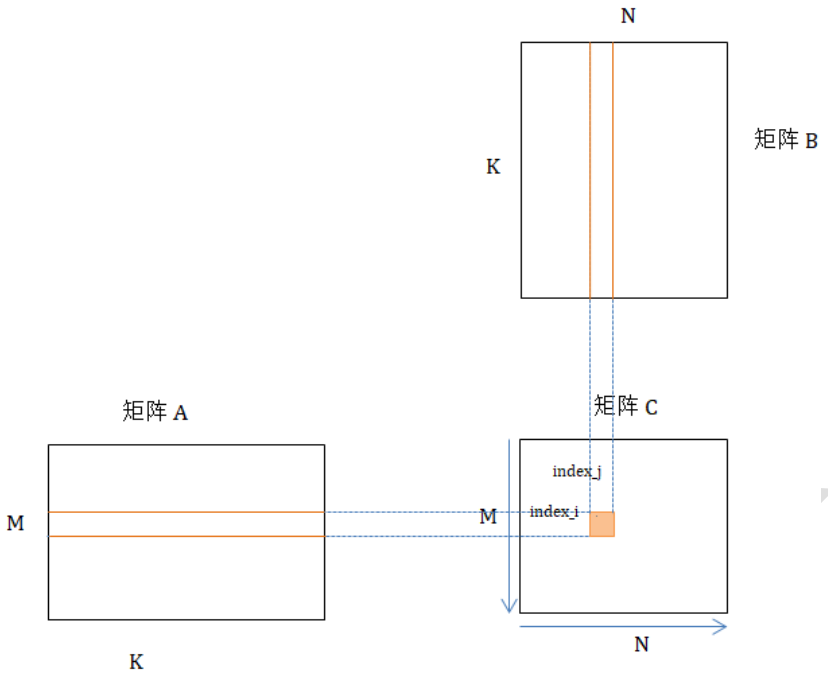


图 1.5 当前线程处理的小矩阵块所需的矩阵 A、矩阵 B 的区域

该 kernel 内部，有一个三层循环，用于计算 C 矩阵 4x4 小块内的值，在 K 方向，每次读取 A 矩阵的 4 个数据（按行）和 B 矩阵的 4 个数据（按列）,直到最内层循环结束为止。每次计算一个 c 值，计算完后再计算其他 c 值。

该代码初步实现了，GEMM 算法到 DCU 上的移植，我们会发现如同 1.1 中分析的那样，A 矩阵、B 矩阵中的数据存在大量的重复读取。我们可以从减少重复数据读取为优化点，进行进一步的优化。在我们将程序往 DCU 上移植时，首先需要考虑的是任务的划分，需要注意串行程序与并行程序的编写的不同之处，学会充分利用 DCU 的硬件资源，来使自己的程序有明显的性能提升。

1.3 使用寄存器实现数据复用

由于 A 矩阵和 B 矩阵的数据存储在全局内存中，访存速度很慢。而寄存器属于片上资源，访存速度极快。因此，在本小节的优化中，我们将从 A 矩阵读取的 4 个数据，B 矩阵读取的 4 个数据以及中间结果全都存储到寄存器中，并且按照 1-1 中的策略，每次读取的 A、B 数据用来进行 4x4 小块内所有位置值的累加更新；接着沿 K 的方向以 depth=8 进行循环展开。4x4 划块，depth=8 GEMM 算法计算如下图所示：

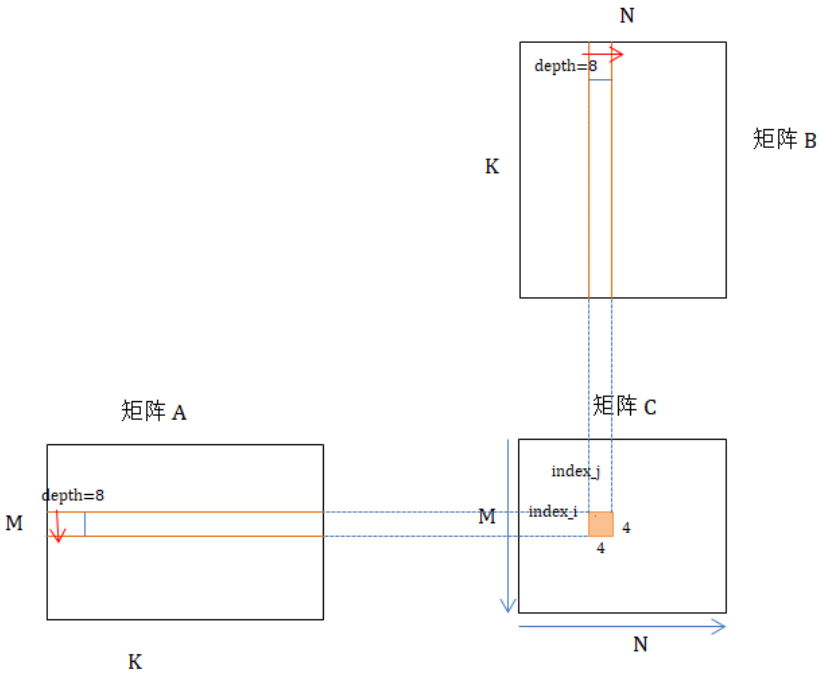


图 1.6 4x4 划块，depth=8 GEMM 算法示意图

下面一个宏 MAC_4x4 主要完成从 A 矩阵、B 矩阵中读取的 4 个数据的乘累加操作，所有的操作均在寄存器中，每次计算会得到 4x4 最终结果的一“层”，直到沿 K 方向循环结束。

```
/* MAC's */
#define MAC(A,B,DST) DST += A*B
#define TYPE_MAC(MULA,MULB,DST) DST = MAC(MULA,MULB,DST);
/* 4x4 micro-tile */
#define MAC_4x4\
    TYPE_MAC(rA[0],rB[0],rC[0]); \
    TYPE_MAC(rA[0],rB[1],rC[1]); \
    TYPE_MAC(rA[0],rB[2],rC[2]); \
    TYPE_MAC(rA[0],rB[3],rC[3]); \
    TYPE_MAC(rA[1],rB[0],rC[4]); \
    TYPE_MAC(rA[1],rB[1],rC[5]); \
    TYPE_MAC(rA[1],rB[2],rC[6]); \
    TYPE_MAC(rA[1],rB[3],rC[7]); \
```

```
TYPE_MAC(rA[2],rB[0],rC[8]); \
TYPE_MAC(rA[2],rB[1],rC[9]); \
TYPE_MAC(rA[2],rB[2],rC[10]); \
TYPE_MAC(rA[2],rB[3],rC[11]); \
TYPE_MAC(rA[3],rB[0],rC[12]); \
TYPE_MAC(rA[3],rB[1],rC[13]); \
TYPE_MAC(rA[3],rB[2],rC[14]); \
TYPE_MAC(rA[3],rB[3],rC[15]); \
```

该宏计算如图 1.7 所示。

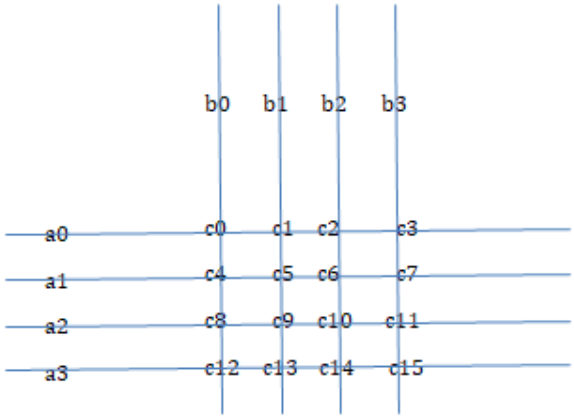


图 1.7 MAC_4x4 计算

也即是：c0=a0xb0, c1=a0xb1, c2=a0xb2, c3=a0xb3，..., c15=a3xb3（其中，a0~a3 表示 rA[0]~rA[3], b0~b3 表示 rB[0]~rB[3], c0~c15 表示 rC[0]~rC[15]），本部分计算使用的是寄存器资源，实现数据的复用，不再频繁地重复读取全局内存中的数据。

每次沿图 1.6 红色箭头的方向，读取 A 矩阵 4 个数据，B 矩阵 4 个数据。计算出 4x4 小矩阵块的一次部分值（可以看做一“层”），然后进行累加，并沿 K 方向进行 depth=8 的循环展开：

```
for(i=0; i<size_k; i+=8)
{
    //iter 0
    rA[0] = *(tmp_a + i*size_m + 0);
    rA[1] = *(tmp_a + i*size_m + 1);
```

```
    rA[2] = *(tmp_a + i*size_m + 2);
    rA[3] = *(tmp_a + i*size_m + 3);
    rB[0] = *(tmp_b + i*size_n + 0);
    rB[1] = *(tmp_b + i*size_n + 1);
    rB[2] = *(tmp_b + i*size_n + 2);
    rB[3] = *(tmp_b + i*size_n + 3);
    MAC_4x4
     //iter 1
    rA[0] = *(tmp_a + (i+1)*size_m + 0);
    rA[1] = *(tmp_a + (i+1)*size_m + 1);
    rA[2] = *(tmp_a + (i+1)*size_m + 2);
    rA[3] = *(tmp_a + (i+1)*size_m + 3);
    rB[0] = *(tmp_b + (i+1)*size_n + 0);
    rB[1] = *(tmp_b + (i+1)*size_n + 1);
    rB[2] = *(tmp_b + (i+1)*size_n + 2);
    rB[3] = *(tmp_b + (i+1)*size_n + 3);
    MAC_4x4
        ...
}
```

循环展开能够增大指令调度的空间，减少循环分支指令的开销，即增大指令流水线的效率。同时，循环展开可以更好地实现数据预取技术。对于每个 4x4 小块的计算，A 矩阵和 B 矩阵的访存由 16K 次减少到 4K 次，减少了 3/4。

该代码利用寄存器实现了数据复用，减少了访存。在该代码实现中，数据读取完后，直接使用该数据进行计算，会存在计算指令等待访存完成的现象，直至读取操作完成，这会产生一定的延迟。我们可以使用数据预取技术将这种延迟隐藏掉。

由前面的章节，我们知道，寄存器访存相比全局内存的访问时间，是非常快的。在开发者今后的优化中，可以将全局内存中的数据读取到寄存器中（利用寄存器的最低延迟和超高带宽的优势），之后如果涉及到数据的复用，避免了频繁

且低效的对全局内存的访问，这是一个很容易想到的优化点。

1.4 数据预取隐藏延迟

在 DCU 中，计算单元和存取单元是不同的单元；在单个线程内，在没有数据依赖的情况下，访存操作和计算单元的计算可以同时进行。这两种不同功能单元操作上的重叠，可以形成流水化，隐藏数据访问的延迟。

在实际使用时，如果原始代码是：读取数据 a0,b0--使用 a0,b0 进行计算，我们可以在这两条语句之间，添加下次计算要使用的数据 a1,b1，它与 a0，b0 的计算没有依赖性，就可以并行执行，抵消掉一部分数据读取的延迟。

在本小节的优化中，为了使用数据预取技术，我们将读取 A 矩阵、B 矩阵数据所用的寄存器的大小都扩大了一倍（即由原来的的 rA[4], rB[4]扩大为 rA[8]与 rB[8], 其中代码中的 TT 为 4）。第一组数据在循环外面进行读取，读取完后，没有直接进行计算，中间隔了下一组数据的读取，再进行乘累加计算。数据预取的代码如下所示。

```
//preload
rA[0] = *(tmp_a + 0);
rA[1] = *(tmp_a + 1);
rA[2] = *(tmp_a + 2);
rA[3] = *(tmp_a + 3);

rB[0] = *(tmp_b + 0);
rB[1] = *(tmp_b + 1);
rB[2] = *(tmp_b + 2);
rB[3] = *(tmp_b + 3);

for(i=1;i<size_k-8;i+=8)
{
    //iter 0
    rA[0+TT] = *(tmp_a + i*size_m + 0);
    rA[1+TT] = *(tmp_a + i*size_m + 1);
```

```
        rA[2+TT] = *(tmp_a + i*size_m + 2);
        rA[3+TT] = *(tmp_a + i*size_m + 3);

        rB[0+TT] = *(tmp_b + i*size_n + 0);
        rB[1+TT] = *(tmp_b + i*size_n + 1);
        rB[2+TT] = *(tmp_b + i*size_n + 2);
        rB[3+TT] = *(tmp_b + i*size_n + 3);
        MAC_4x4

        //iter 1
        rA[0] = *(tmp_a + (i+1)*size_m + 0);
        rA[1] = *(tmp_a + (i+1)*size_m + 1);
        rA[2] = *(tmp_a + (i+1)*size_m + 2);
        rA[3] = *(tmp_a + (i+1)*size_m + 3);

        rB[0] = *(tmp_b + (i+1)*size_n + 0);
        rB[1] = *(tmp_b + (i+1)*size_n + 1);
        rB[2] = *(tmp_b + (i+1)*size_n + 2);
        rB[3] = *(tmp_b + (i+1)*size_n + 3);
        MAC_4x4_BLK
        ...
    }
```

其中，MAC_4x4_BLK 与 MAC_4x4 的定义类似，主要的作用就是完成从 A 矩阵、B 矩阵中读取的 4 个数据的乘累加操作，所有的操作均在寄存器中，每次计算会得到 4x4 最终结果的一“层”，直到沿 K 方向循环结束。

```
#define MAC_4x4_BLK\
   TYPE_MAC(rA[0+TT],rB[0+TT],rC[0]); \
   TYPE_MAC(rA[0+TT],rB[1+TT],rC[1]); \
   TYPE_MAC(rA[0+TT],rB[2+TT],rC[2]); \
```

```
TYPE_MAC(rA[0+TT],rB[3+TT],rC[3]); \
TYPE_MAC(rA[1+TT],rB[0+TT],rC[4]); \
TYPE_MAC(rA[1+TT],rB[1+TT],rC[5]); \
TYPE_MAC(rA[1+TT],rB[2+TT],rC[6]); \
TYPE_MAC(rA[1+TT],rB[3+TT],rC[7]); \
TYPE_MAC(rA[2+TT],rB[0+TT],rC[8]); \
TYPE_MAC(rA[2+TT],rB[1+TT],rC[9]); \
TYPE_MAC(rA[2+TT],rB[2+TT],rC[10]); \
TYPE_MAC(rA[2+TT],rB[3+TT],rC[11]); \
TYPE_MAC(rA[3+TT],rB[0+TT],rC[12]); \
TYPE_MAC(rA[3+TT],rB[1+TT],rC[13]); \
TYPE_MAC(rA[3+TT],rB[2+TT],rC[14]); \
TYPE_MAC(rA[3+TT],rB[3+TT],rC[15]); \
```

在循环开始前，首先进行一次数据的预读取，放到 rA[0...3], rB[0...3]中，然后进入循环后，将下一次用到的数据进行读取，放到 rA[4...7], rB[4...7]中，接着执行 MAC_4x4 或者 MAC_4x4_BLK。最后一个 depth=8，放到循环外执行。

这样的操作，可以使预取下一次计算所需要的的数据的指令和本次计算的过程指令进行交叠，减少计算指令等待访存完成的现象，可以将访存的延迟给隐藏掉一部分。在开发者实际编程过程中，可以通过这种策略加上循环展开，提高程序的指令级的并行（原因在于通过循环展开编译器可以在更长的无数据依赖的指令序列中对指令进行排序，同时减少循环索引和部分存储器地址的计算），使执行更加地高效，这同样是一个很容易想到的常见的优化点。

在实际不同的应用中，采用数据预取时，会存在寄存器使用得比较多，可能导致性能下降的情况。

1.5 使用 LDS 进行访存优化

对于采用分块的算法，当前数据块同行、同列的数据块与当前数据块之间同样存在对 global Memory 的重复访存，此时我们可以借助前面介绍的片上存储 LDS 完成对数据的搬移，然后从 LDS 上将数据读取到寄存器中。由于 LDS 的访存速度比 global Memory 的速度快得多，因此这种搬移数据的方式有助于性能的

提升（数据块之间对 global Memory 的重复访问，转换为对 LDS 的重复访问）。

也就是说，采用本小节的优化方法后，数据的读取方式由：

global Memory->寄存器

变换成：

global Memory->LDS->寄存器

使用这种优化策略后，在 depth=8 的情况下，每次将 A 矩阵 4xdepth、B 矩阵 4xdepth 个数据读取到 LDS 中，同步后，直接将数据从 LDS 中读取到寄存器中进行运算。4x4 划块，depth=8、数据预取并采用 lds 的 GEMM 算法计算如图 1.8 所示。

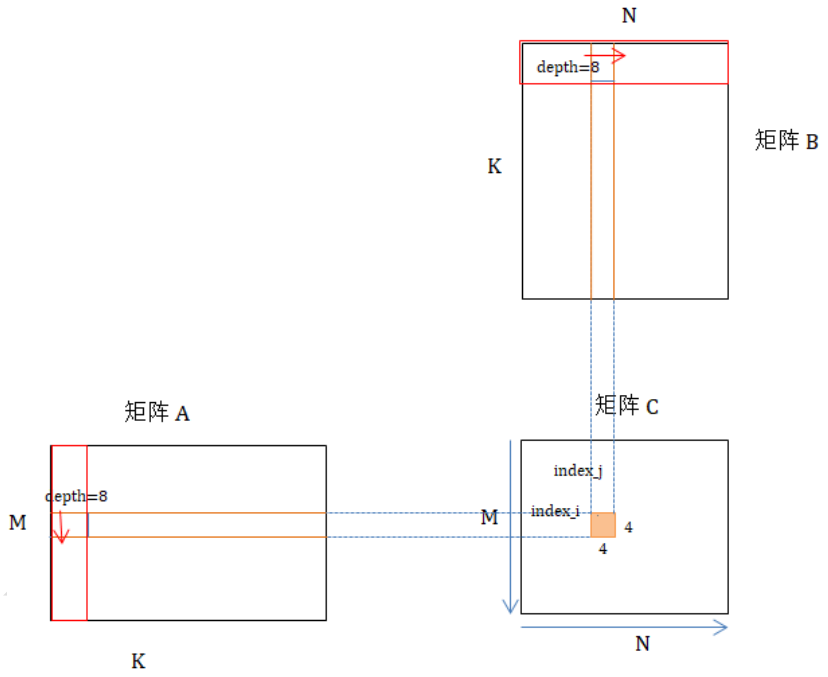


图 1.8 使用 LDS 进行数据的搬移

下面我们对该小节主要的代码进行分析。

global memory->LDS 的数据读取

```
/* allocate local memory */
__shared__ double localMemory[LDS_NUM_ELEMENTS];

unsigned int serial = threadIdx.x;   // 线程号
```

```
double *local_write_A =    localMemory + serial * 2;
double *local_write_B =    localMemory + serial * 2 + LDS_OFFSET_B;
...
for(i=0;i<size_k;i+=8)
{
    //global -> lds
    global_a0 = *(global_address_A + 0); //global read A
    global_a1 = *(global_address_A + 1);
    global_b0 = *(global_address_B + 0); //global read B
    global_b1 = *(global_address_B + 1);

    global_address_A += DEPTH * size_m;
    global_address_B += DEPTH * size_n;

    //write lds
    *(local_write_A + 0) = global_a0;
    *(local_write_A + 1) = global_a1;
    *(local_write_B + 0) = global_b0;
    *(local_write_B + 1) = global_b1;

    __syncthreads();
     ...
}
```

这段代码中，localMemory 是定义的 LDS。在该 GEMM kernel 的实现中，每次读取 A 矩阵 Mxdepth=64x8， B 矩阵 Nxdepth=64x8 个数据。因此每个线程只需负责 2 个 A 矩阵数据，2 个 B 矩阵数据，即可完成将 global Memory 中的本次 depth=8 区域内数据到 LDS 中的写入。（ global_address_A，global_address_B 是本次读取 global Memory 的数据时，数据的首地址； LDS_NUM_ELEMENTS 等于 1024，LDS_OFFSET_B 等于 512。）

lds 数据到寄存器中的读取

```
/* allocate local memory */
 __shared__ double localMemory[LDS_NUM_ELEMENTS];

unsigned int serial = threadIdx.x;//线程号
...
unsigned int lrj = (serial >> 4);
unsigned int lri = (serial & 15);

double *local_read_A = localMemory + lri * 4;
double *local_read_B = localMemory + lrj * 4 + LDS_OFFSET_B;
...
for(i=0;i<size_k;i+=8)
{
        ...
      //write lds
      *(local_write_A + 0) = global_a0;
      *(local_write_A + 1) = global_a1;
      *(local_write_B + 0) = global_b0;
      *(local_write_B + 1) = global_b1;
      __syncthreads();
      //preload
      ...

      local_read_A += MT0I;
      local_read_B += MT1J;
      //iter 0
      rA[0+TT] = *(local_read_A + 0);
      rA[1+TT] = *(local_read_A + 1);
```

```
        rA[2+TT] = *(local_read_A + 2);
        rA[3+TT] = *(local_read_A + 3);

        rB[0+TT] = *(local_read_B + 0);
        rB[1+TT] = *(local_read_B + 1);
        rB[2+TT] = *(local_read_B + 2);
        rB[3+TT] = *(local_read_B + 3);
        ...
    }
```

从 LDS 中读取数据，与前几个小节从 global memory 中直接读取的逻辑是类似的，区别在于原来列宽为 size_m，size_n，现在变成了 MT0I 和 MT1J，这里都是 64。接下来的操作，从 LDS 中读取数据进行数据预取。1.1.4 小节是对 global Memory 的数据进行预取，而本小节我们进一步加入了 LDS 的数据预取流水，从而通过将计算和共享内存操作重叠来进一步隐藏共享内存的延迟，同时减少计算中等待数据的流水线停顿。

由于我们是将数据完全搬移到 LDS 中之后，才进行后续的读取到寄存器以及计算的流程，这种方式和 1.1 的第 4 节介绍的一样，同样存在访存延迟的现象。

DCU 可以通过 block 内部的 wavefront 之间的切换隐藏内存访问和计算的延迟；通过 block 之间的切换隐藏线程块内同步的延迟。由于寄存器和 LDS 资源的限制，单个 CU 能驻留的活跃 block 的数量是有限的，因此我们无法通过增加每个 CU 上可并发的 block 数量来进一步去提升效率。类似于单个线程内的并行和并发，此时我们需要寻求单个 block 内的更多的并行和并发机制。

在上面代码中，本次循环内用到的数据从 global Memory->LDS 读取完成后（需要进行同步，而主循环体内的并行和并发受限于块内同步），才进行 LDS->寄存器的读取以及后续的计算流程。因此，我们可以通过将 LDS 的存储空间加倍，一半用于进行读数据的操作，一半进行写数据的操作，如图 1.9 所示。在本次循环内预取下一次循环所用到的数据，则可以减少块内同步带来的开销，有些书籍或者文章中也称为"双缓冲"技术。

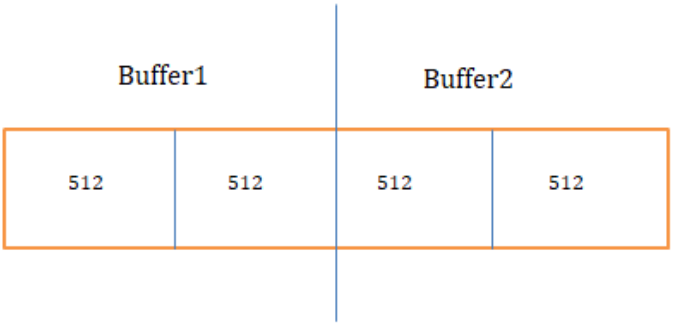


图 1.9 lds 双 buffer 的使用

使用双缓冲技术后，整个优化后的算法实际上存在了两级数据预取，第一级就是 global Memory->LDS，第二级是从 LDS->寄存器。两级数据预取形成了两级流水，可以将 kernel 内的延迟有效地隐藏，提高程序的执行效率。优化后的算法计算如图所示，其中蓝色框出来的区域就是下一次循环步需要使用的数据，需要被预取到 LDS 中。

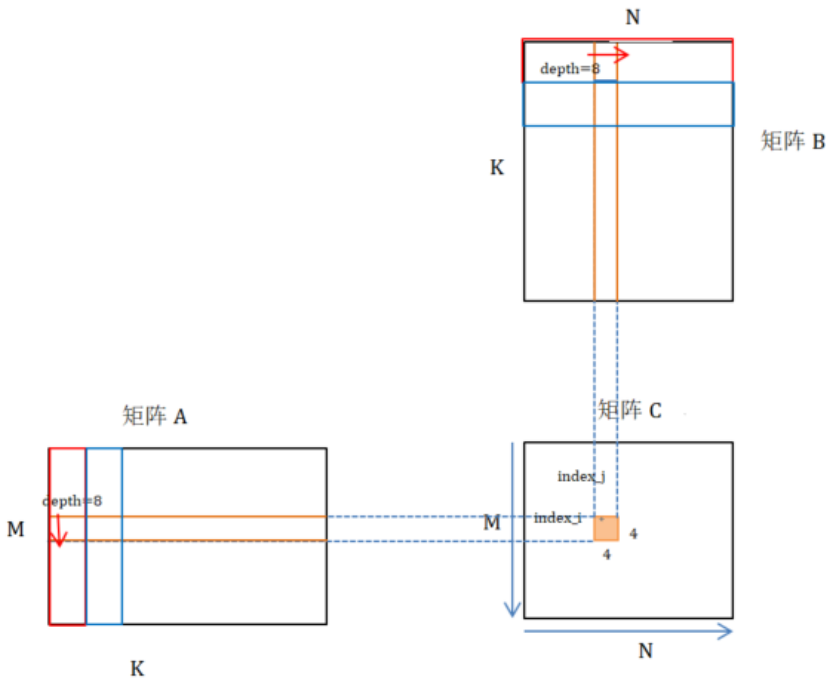


图 1.10 lds “双缓冲”技术的使用

如下代码，进行读写 buffer 的切换，例如本次使用 Buffer1 读，往 Buffer2 中写；则下次使用 Buffer2 读，往 Buffer1 中写：

```
...
#define LDS_NUM_ELEMENTS 2048
#define LDS_OFFSET_B 512
```

```
#define LDS_OFFSET_BLK 1024
#define DEPTH 8
#define MT0I 64
#define MT1J 64
...

/* allocate local memory */
__shared__ double localMemory[LDS_NUM_ELEMENTS];
//global -> lds
global_a0 = *(global_address_A + 0); //global read A
global_a1 = *(global_address_A + 1);
global_b0 = *(global_address_B + 0); //global read B
global_b1 = *(global_address_B + 1);
global_address_A += DEPTH * size_m;
global_address_B += DEPTH * size_n;
//write lds
*(local_write_A + 0) = global_a0;
*(local_write_A + 1) = global_a1;
*(local_write_B + 0) = global_b0;
*(local_write_B + 1) = global_b1;
lwa = (lwa + LDS_OFFSET_BLK) % LDS_NUM_ELEMENTS;
lwb = (lwb + LDS_OFFSET_BLK) % LDS_NUM_ELEMENTS;
// switch two lds write address for the next write process
local_write_A =    localMemory + lwa;
local_write_B =    localMemory + lwb;
```

代码中可以控制好 global 到 lds 的数据搬运时间,除了第一个是全部搬运完再进行计算，剩余循环的数据搬运可以和其他指令穿插进行，便于延时控制，数据的读取都是从 LDS 中直接读取的，LDS 在线程之间公用。

在该代码中，采用两块 LDS 进行交替读取数据的方式。在使用 LDS 进行性

能优化时，我们还需要考虑到的一个问题是，减少 bank 冲突。

开发者在使用，LDS 和寄存器的优化策略时，需要注意避免单个线程内资源的过多使用，因为如果使用的过多，会出现可同时并行调度执行的线程块数量下降的情况，导致性能的损失。

1.6 减少 bank 冲突

在 LDS 使用时，需要特别注意的一个问题是 bank 冲突。在前面的章节，有关于 bank 冲突比较详细的介绍，简单来说就是：对于同一个 wavefront 里面的线程，同时访问同一个 bank 内的不同地址时，会出现串行读取的情况，导致 LDS 访存效率的降低。

下面我们首先对 lds 访存中存在的 bank 冲突的情况进行分析。在该代码中，我们使用 1 个线程块，并且该线程块只有 x 维度不为 1，线程块中有 256 个线程。每个线程，映射到对应的 4x4 的小矩阵块，数据的读取是连续读取的，由于 LDS 中有 32 个 bank，假设线程 0 在 bank0 上，针对 bank 冲突的情况，我们继续进行分析与优化。

```
unsigned int serial    = threadIdx.x;    // 1D  线程号
unsigned int lrj = (serial >> 4); //  当前矩阵块左上角元素所在 C 矩阵的列
unsigned int lri = (serial & 15); //  当前矩阵块左上角元素所在 C 矩阵的行
...
unsigned int lra = lri * 4;
unsigned int lrb = lrj * 4 + LDS_OFFSET_B;
double *local_read_A = localMemory + lra;
double *local_read_B = localMemory + lrb;
...
rA[0] = *(local_read_A + 0);
rA[1] = *(local_read_A + 1);
rA[2] = *(local_read_A + 2);
rA[3] = *(local_read_A + 3);
rB[0] = *(local_read_B + 0);
rB[1] = *(local_read_B + 1);
```

```
rB[2] = *(local_read_B + 2);
rB[3] = *(local_read_B + 3);
```

在此种情况下，在一个 wavefront 64 个线程内部，线程对应的 4x4 小块的行列索引为：

thread0:(0,0) thread16:(0,1) thread32:(0,2) thread48:(0,3)

thread1:(1,0) thread17:(1,1) thread33:(1,2) thread49:(1,3)

thread2:(2,0) thread18:(2,1) thread34:(2,2) thread50:(3,3)

...

thread15:(15, 0) thread31:(15,1) thread47:(15,2) thread63:(3,3)

则：

thread0 连续访问 A 矩阵 4 个数据，占用 bank0-bank7, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread1 连续访问 A 矩阵 4 个数据，占用 bank8-bank15, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread2 连续访问 A 矩阵 4 个数据，占用 bank16-bank23, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread3 连续访问 A 矩阵 4 个数据，占用 bank24-bank31, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread4 连续访问 A 矩阵 4 个数据，占用 bank0-bank7, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread5 连续访问 A 矩阵 4 个数据，占用 bank8-bank15, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread6 连续访问 A 矩阵 4 个数据，占用 bank16-bank23, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread7 连续访问 A 矩阵 4 个数据，占用 bank24-bank31, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread8 连续访问 A 矩阵 4 个数据，占用 bank0-bank7, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread9 连续访问 A 矩阵 4 个数据，占用 bank8-bank15, 连续访问 B 矩阵 4 个数

据，占用 bank0-bank7；

thread10 连续访问 A 矩阵 4 个数据，占用 bank16-bank23, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread11 连续访问 A 矩阵 4 个数据，占用 bank24-bank31, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread12 连续访问 A 矩阵 4 个数据，占用 bank0-bank7, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread13 连续访问 A 矩阵 4 个数据，占用 bank8-bank15, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread14 连续访问 A 矩阵 4 个数据，占用 bank16-bank23, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

thread15 连续访问 A 矩阵 4 个数据，占用 bank24-bank31, 连续访问 B 矩阵 4 个数据，占用 bank0-bank7；

...

thread63 连续访问 A 矩阵 4 个数据，占用 bank24-bank31, 连续访问 B 矩阵 4 个数据，占用 bank24-bank31；

从该分析可以看出，一个 wavefront 内部，对 lds 中 A 的访问存在 4 路 bank 冲突，对 B 的访问不存在 bank 冲突（（0~15、16~31、32~47 以及 48~63 号线程，可以分为 4 组，组之间需要的 local_read_A 的数据是相同的，组之内需要的 local_read_B 的数据是相同的）。

为了减少 bank 冲突，我们改变从 LDS 中进行数据读取的方式，每个线程不采用连续读取 4 个数据的方式，而是首先连续读取 2 个数据，然后间隔 32 个数，再连续读取 2 个数据：

```
//iter 0
#define SIZE_HALF_A 32
#define SIZE_HALF_B 32
...
rA[0+TT] = *(local_read_A + 0);
rA[1+TT] = *(local_read_A + 1);
```

```
rA[2+TT] = *(local_read_A + 0 + SIZE_HALF_A);
rA[3+TT] = *(local_read_A + 1 + SIZE_HALF_A);
rB[0+TT] = *(local_read_B + 0);
rB[1+TT] = *(local_read_B + 1);
rB[2+TT] = *(local_read_B + 0 + SIZE_HALF_B);
rB[3+TT] = *(local_read_B + 1 + SIZE_HALF_B);
```

改写之后，我们进行分析如下：

thread0 访问 A 矩阵 4 个数据，占用 bank0-bank3，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread1 访问 A 矩阵 4 个数据，占用 bank4-bank7，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread2 访问 A 矩阵 4 个数据，占用 bank8-bank11，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread3 访问 A 矩阵 4 个数据，占用 bank12-bank15，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread4 访问 A 矩阵 4 个数据，占用 bank16-bank19，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread5 访问 A 矩阵 4 个数据，占用 bank20-bank23，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread6 访问 A 矩阵 4 个数据，占用 bank24-bank27，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread7 访问 A 矩阵 4 个数据，占用 bank28-bank31，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread8 访问 A 矩阵 4 个数据，占用 bank0-bank3，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread9 访问 A 矩阵 4 个数据，占用 bank4-bank7，访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread10 访问 A 矩阵 4 个数据，占用 bank8-bank11，访问 B 矩阵 4 个数据，占用 bank0-bank3；

开发者社区

thread11 访问 A 矩阵 4 个数据，占用 bank12-bank15, 访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread12 访问 A 矩阵 4 个数据，占用 bank16-bank19, 访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread13 访问 A 矩阵 4 个数据，占用 bank20-bank23, 访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread14 访问 A 矩阵 4 个数据，占用 bank24-bank27, 访问 B 矩阵 4 个数据，占用 bank0-bank3；

thread15 访问 A 矩阵 4 个数据，占用 bank28-bank31, 访问 B 矩阵 4 个数据，占用 bank0-bank3；

...

thread63 访问 A 矩阵 4 个数据，占用 bank28-bank31, 访问 B 矩阵 4 个数据，占用 bank12-bank15；

通过这种数据读取的方式，我们可以将 bank 冲突从 4 路减少到 2 路（0~15、16~31、32~47 以及 48~63 号线程，可以分为 4 组，组之间需要的 local_read_A 的数据是相同的，组之内需要的 local_read_B 的数据是相同的，由修改后的代码的分析可以看出，0~15 号线程读取 bank0~bank31 的数据重复了一次，因此可以认为有 2 路 bank 冲突）。

bank 冲突是进行深度优化时，一个比较常见的优化点，开发者在使用 LDS 进行程序的编写后，对自己的程序做静态的分析，以一个 wavefront 内线程的访存为例，看看当前设计的程序的 bank 冲突的情况，可以根据当前冲突的情况，改变线程读取 LDS 的方式，如同本节这样；或者改变 LDS 的数据组织方式，比如对二维数组的数据最后做一列填充。

2 并行规约问题

在向量中执行满足交换律和结合律的运算，被称为归约问题，并行归约问题是这种运算的并行执行。并行归约是一种最常见的并行模式，并且是许多并行算法中的一个关键运算。

在串行编程中，我们常见的一个问题就是一组特别多数字通过计算变成一个数字，比如加法，也就是求这一组数据的和，或者乘法。以加法举例，因为这种

计算有结合性和交换性，可以采用并行归约的方法处理他们。基本方法是这样的：

1. 将输入向量划分到更小的数据块中
2. 用一个线程计算一个数据块的部分和
3. 对每个数据块的部分和再求和得到最终的结果

并行归约的一个常用方法是使用迭代成对实现。一个数据块只包含一对元素，并且一个线程对这两个元素求和产生一个局部结果。然后，这些局部结果在最初的输入向量中就地保存。这些新值被作为下一次迭代求和的输入值。因为输入值的数量在每一次迭代后会减半，当输出向量的长度达到 1 时，最终的和就已经被计算出来了。

成对的划分常见的方法有以下两种：

1. 相邻配对：元素与他们相邻的元素配对

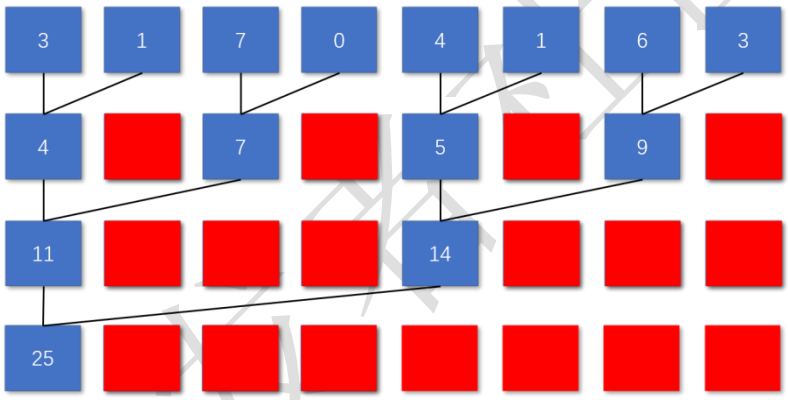


图 2.1 相邻配对示意图

2. 交错配对：元素与一定距离的元素配对

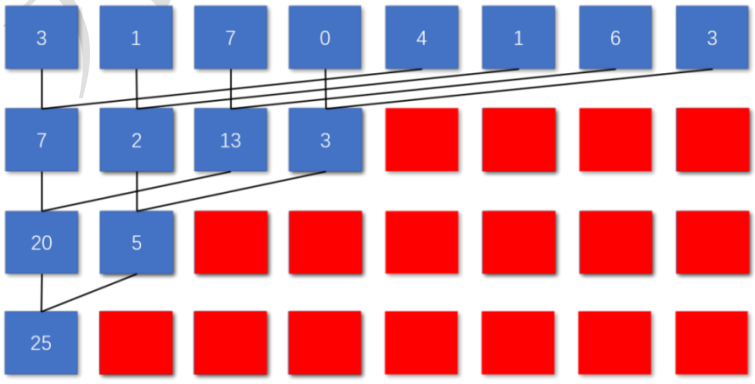


图 2.2 交错配对示意图

尽管以上代码实现的是加法，但任何满足交换律和结合律的运算都可以代替加法。例如，通过调用 max 代替求和运算，就可以计算输入向量中的最大值。其

他有效运算的例子有最小值、平均值和乘积。

在本节里，会实现多个不同的并行归约核函数，并且将测试不同的实现是如何影响内核性能的。

并行归约中的分化

下图所示的是相邻配对方法的内核实现流程。每个线程将相邻的两个元素相加产生部分和。

在这个内核里，有两个全局内存数组：一个大数组用来存放整个数组，进行归约；另一个小数组用来存放每个线程块的部分和。每个线程块在数组的一部分上独立地执行操作。循环中迭代一次执行一个归约步骤。归约是在就地完成的，这意味着在每一步，全局内存里的值都被部分和替代。__syncthreads 语句可以保证，线程块中的任一线程在进入下一次迭代之前，在当前迭代里每个线程的所有部分和都被保存在了全局内存中。进入下一次迭代的所有线程都使用上一步产生的数值。在最后一个循环以后，整个线程块的和被保存进全局内存中。

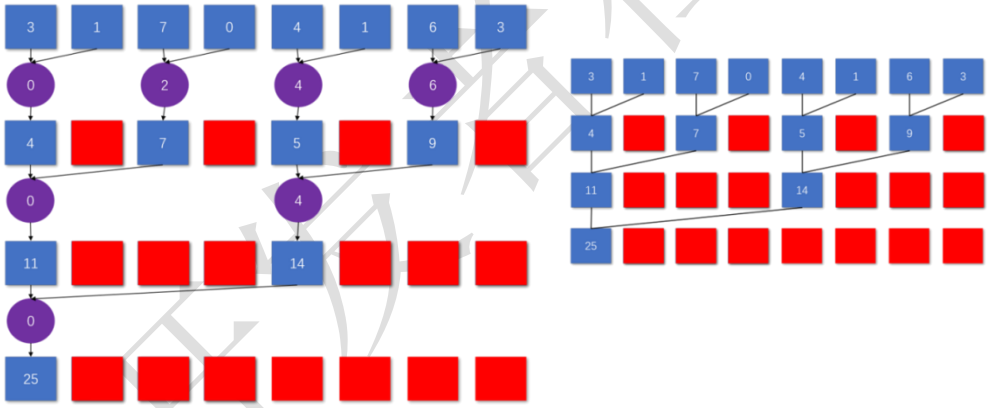


图 2.3 相邻配对方法的内核实现流程示意图

```
__global__ void reduceNeighbored(int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;
    // boundary check
    if (idx >= n) return;
```

```
    // in-place reduction in global memory
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if ((tid % (2 * stride)) == 0) {
            idata[tid] += idata[tid + stride];
        }
        // synchronize within threadblock
        __syncthreads();
    }
    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

两个相邻元素间的距离被称为跨度，初始化均为 1。在每一次归约循环结束后，这个间隔就被乘以 2。在第一次循环结束后，idata（全局数据指针）的偶数元素将会被部分和替代。在第二次循环结束后，idata 的每四个元素将会被新产生的部分和替代。因为线程块间无法同步，所以每个线程块产生的部分和被复制回了主机，并在主机端串行求和， 如图 1.14 所示。

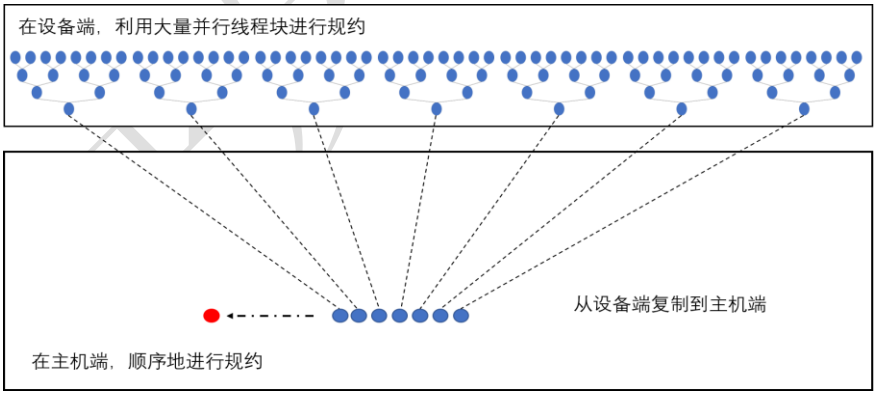


图 1.14 操作思路示意图

```
int main(int argc, char **argv) {
    // set up device
    int dev = 0;
    bool bResult = false;
    // initialization
```

```
int size = 1 << 24; // total number of elements to reduce
printf("       with array size %d    ", size);
// execution configuration
int blocksize = 512;      // initial block size
if (argc > 1) {
      blocksize = atoi(argv[1]);      // block size from command line argument
}
dim3 block(blocksize, 1);
dim3 grid((size + block.x - 1) / block.x, 1);
printf("grid %d block %d\n", grid.x, block.x);
hipEvent_t start_time, stop_time;
hipEventCreate(&start_time);
hipEventCreate(&stop_time);
// allocate host memory
size_t bytes = size * sizeof(int);
int *h_idata = (int *) malloc(bytes);
int *h_odata = (int *) malloc(grid.x * sizeof(int));
int *tmp = (int *) malloc(bytes);
float time_elapsed = 1.0f;
// initialize the array
for (int i = 0; i < size; i++) {
      // mask off high 2 bytes to force max number to 255
      h_idata[i] = (int) (rand() & 0xFF);
}
memcpy(tmp, h_idata, bytes);
//double iStart, iElaps;
int gpu_sum = 0;
// allocate device memory
int *d_idata = NULL;
```

```
    int *d_odata = NULL;
    hipMalloc((void **) &d_idata, bytes);
    hipMalloc((void **) &d_odata, grid.x * sizeof(int));
    // cpu reduction
    hipEventRecord(start_time, NULL);
    int cpu_sum = recursiveReduce(tmp, size);
    hipEventRecord(stop_time, NULL);
    hipEventElapsedTime(&time_elapsed, start_time, stop_time);
    printf("cpu reduce elapsed %f sec cpu_sum: %d\n", time_elapsed, cpu_sum);

    // kernel 1: reduceNeighbored
    hipMemcpy(d_idata, h_idata, bytes, hipMemcpyHostToDevice);
    hipDeviceSynchronize();
    hipEventRecord(start_time, NULL);
    hipLaunchKernelGGL(reduceNeighbored, grid, block, 0, 0, d_idata, d_odata,
size);
    hipDeviceSynchronize();
    hipEventRecord(stop_time, NULL);
    hipEventElapsedTime(&time_elapsed, start_time, stop_time);
    hipMemcpy(h_odata, d_odata, grid.x * sizeof(int), hipMemcpyDeviceToHost);
    gpu_sum = 0;
    for (int i = 0; i < grid.x; i++) gpu_sum += h_odata[i];
    printf("gpu Neighbored    elapsed %f sec gpu_sum: %d <<<grid %d block "
           "%d>>>\n", time_elapsed, gpu_sum, grid.x, block.x);
    // kernel 2: reduceNeighbored with less divergence
    hipMemcpy(d_idata, h_idata, bytes, hipMemcpyHostToDevice);
    hipDeviceSynchronize();
    hipEventRecord(start_time, NULL);
    hipLaunchKernelGGL(reduceNeighboredLess, grid, block, 0, 0, d_idata, d_odata,
```

```
size);
    hipDeviceSynchronize();
    hipEventRecord(stop_time, NULL);
    hipEventElapsedTime(&time_elapsed, start_time, stop_time);
    hipMemcpy(h_odata, d_odata, grid.x * sizeof(int), hipMemcpyDeviceToHost);
    gpu_sum = 0;
    for (int i = 0; i < grid.x; i++) gpu_sum += h_odata[i];
    printf("gpu Neighbored2 elapsed %f sec gpu_sum: %d <<<grid %d block "
           "%d>>>\n", time_elapsed, gpu_sum, grid.x, block.x);
    // kernel 3: reduceInterleaved
    hipMemcpy(d_idata, h_idata, bytes, hipMemcpyHostToDevice);
    hipDeviceSynchronize();
    hipEventRecord(start_time, NULL);
    hipLaunchKernelGGL(reduceInterleaved, grid, block, 0, 0, d_idata, d_odata, size);
    hipDeviceSynchronize();
    hipEventRecord(stop_time, NULL);
    hipEventElapsedTime(&time_elapsed, start_time, stop_time);
    hipMemcpy(h_odata, d_odata, grid.x * sizeof(int), hipMemcpyDeviceToHost);
    gpu_sum = 0;
    for (int i = 0; i < grid.x; i++) gpu_sum += h_odata[i];
    printf("gpu Interleaved elapsed %f sec gpu_sum: %d <<<grid %d block "
           "%d>>>\n", time_elapsed, gpu_sum, grid.x, block.x);
    hipEventDestroy(start_GPU);
    hipEventDestroy(stop_GPU);
    // free host memory
    free(h_idata);
    free(h_odata);
    // free device memory
    hipFree(d_idata);
```

```
    hipFree(d_odata);
    // reset device
    hipDeviceReset();
    // check the results
    bResult = (gpu_sum == cpu_sum);
    if (!bResult) printf("Test failed!\n")
    return 0;
}
```

reduceNeighbored 运行会得出结果：

```
gpu Neighbored    elapsed 1.192479 sec
```

改善并行归约的分化

测试核函数 reduceNeighbored，并注意以下条件表达式：

```
if ((tid%(2*stride))=0)
```

因为上述语句只对偶数 ID 的线程为 true，所以这会导致很高的线程束分化。在并行归约的第一次迭代中，只有 ID 为偶数的线程执行这个条件语句的主体，但是所有的线程都必须被调度。在第二次迭代中，只有四分之一的线程是活跃的，但是所有的线程仍然都必须被调度。通过重新组织每个线程的数组索引来强制 ID 相邻的线程执行求和操作，线程束分化就能被归约了。下图展示了这种实现。和之前函数相比，部分和的存储位置并没有改变，但是工作线程已经更新了。

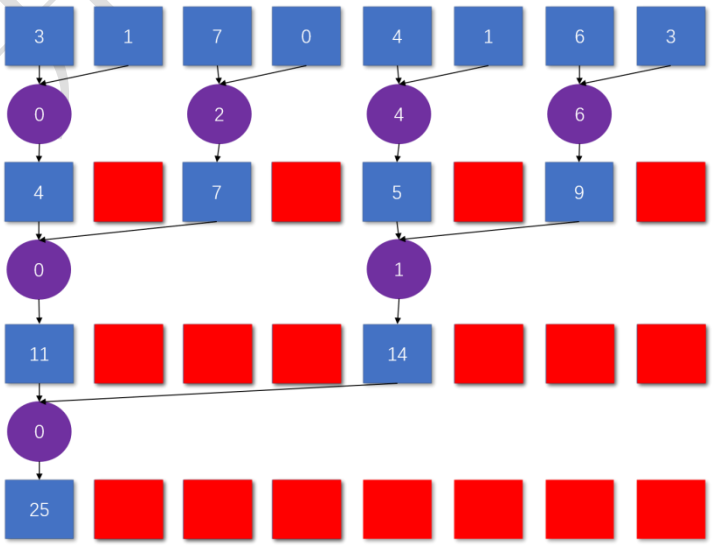


图 2.5 改善相邻规约示意图

修改之后的内核代码：

```
__global__ void reduceNeighboredLess(int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;
    // boundary check
    if (idx >= n) return;
    // in-place reduction in global memory
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        // convert tid into local array index
        int index = 2 * stride * tid;
        if (index < blockDim.x) {
            idata[index] += idata[index + stride];
        }
        // synchronize within threadblock
        __syncthreads();
    }
    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

对于一个有 512 个线程的块来说，前 4 个线程束执行第一轮归约，剩下 4 个线程束什么也不做。在第二轮里，前 2 个线程束执行归约，剩下 6 个线程束什么也不做。因此，这样就彻底不存在分化了。在最后五轮中，当每一轮的线程总数小于线程束的大小时，分化就会出现。

在主函数里调用基准内核之后，会产生以下结果：

```
gpu Neighbored2 elapsed 1.103999 sec
```

结果总结如下，原来的内核在每个线程束里执行的指令数是新内核的两倍多，

它是原来实现高分化的一个指示器；新的实现拥有更高的加载吞吐量，因为虽然 I/O 操作数量相同，但是其耗时更短：

交错配对的归约

与相邻配对方法相比，交错配对方法颠倒了元素的跨度。初始跨度是线程块大小的一半，然后在每次迭代中减少一半（如下图所示）。在每次循环中，每个线程对两个被当前跨度隔开的元素进行求和，以产生一个部分和。与之前方法相比，交错归约的工作线程没有变化。但是，每个线程在全局内存中的加载/存储位置是不同的。

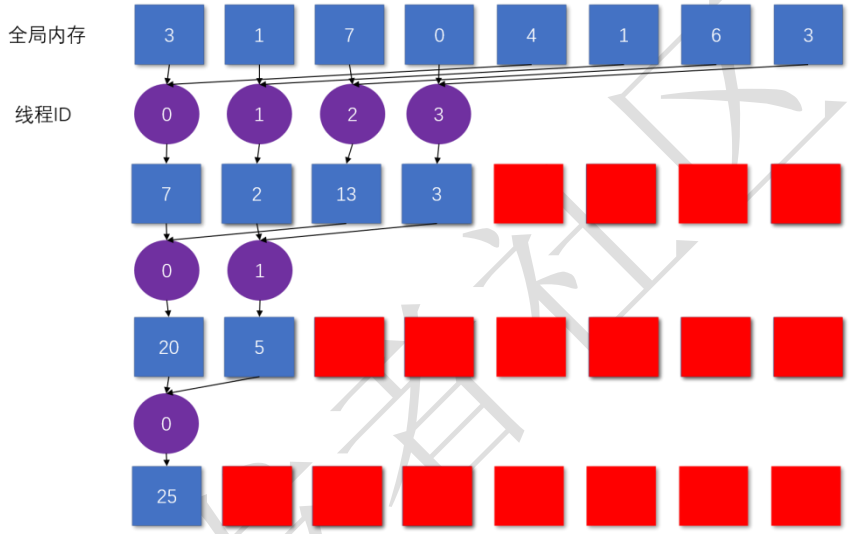


图 2.6 改善交错规约示意图

交错归约的内核代码如下所示：

```
__global__ void reduceInterleaved(int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;
    // boundary check
    if (idx >= n) return;
    // in-place reduction in global memory
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1){
```

```
        if (tid < stride) {
            idata[tid] += idata[tid + stride];
        }
        __syncthreads();
    }
    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

注意核函数中的下述语句，两个元素间的跨度被初始化为线程块大小的一半，然后在每次循环中减少一半：

```
for (int stride = blockDim.x / 2; stride > 0; stride >>= 1)
```

下面的语句在第一次迭代时强制线程块中的前半部分线程执行求和操作，第二次迭代时是线程块的前四分之一，以此类推。

运行结果为：

```
gpu Interleaved elapsed 0.818560 sec
```

交错实现比第一个实现快了 1.46 倍，比第二个实现快了 1.39 倍。这种性能的提升主要是由 reduceInterleaved 函数里的全局内存加载/存储模式导致的。在其他章节会介绍更多有关于全局内存加载/存储模式对内核性能的影响。reduceInterleaved 函数和 reduceNeighboredLess 函数维持相同的线程束分化。

该表总结了所有结果：

内核	时间（s）
相邻（分化）	1.192479
相邻（无分化）	1.103999
交错	0.818560

3 全局内存合并访问

在 DCU 编程中如何安排对全局内存的读写方式是影响程序速度的一个关键点，读者可以先利用下面的代码进行一次带宽测试实验。

```
#include <stdio.h>
```

```
#include <assert.h>
#include <hip/hip_runtime.h>
// Convenience function for checking hip runtime API results
// can be wrapped around any runtime API call. No-op in release builds.
inline
hipError_t checkhip(hipError_t result) {
#if defined(DEBUG) || defined(_DEBUG)
        if (result != hipSuccess) {
            fprintf(stderr, "hip Runtime Error: %s\n", hipGetErrorString(result));
            assert(result == hipSuccess);
        }
#endif
        return result;
}
template<typename T>
__global__ void offset(T *a, int s) {
        int i = blockDim.x * blockIdx.x + threadIdx.x + s;
        a[i] = a[i] + 1;
}
template<typename T>
__global__ void stride(T *a, int s) {
        int i = (blockDim.x * blockIdx.x + threadIdx.x) * s;
        a[i] = a[i] + 1;
}
template<typename T>
void runTest(int deviceId, int nMB) {
        int blockSize = 256;
        float ms;
        T *d_a;
```

```
hipEvent_t startEvent, stopEvent;
int n = nMB * 1024 * 1024 / sizeof(T);
// NB:    d_a(33*nMB) for stride case
checkhip(hipMalloc(&d_a, n * 33 * sizeof(T)));
checkhip(hipEventCreate(&startEvent));
checkhip(hipEventCreate(&stopEvent));
printf("Offset, Bandwidth (GB/s):\n");
offset<<<n / blockSize, blockSize >>> (d_a, 0); // warm up
for (int i = 0; i <= 32; i++) {
        checkhip(hipMemset(d_a, 0, n * sizeof(T)));
        checkhip(hipEventRecord(startEvent, 0));
        offset<<<n / blockSize, blockSize >>> (d_a, i);
        checkhip(hipEventRecord(stopEvent, 0));
        checkhip(hipEventSynchronize(stopEvent));
        checkhip(hipEventElapsedTime(&ms, startEvent, stopEvent));
        printf("%d, %f\n", i, 2 * nMB / ms);
}
printf("\n");
printf("Stride, Bandwidth (GB/s):\n");
stride<<<n / blockSize, blockSize >>> (d_a, 1); // warm up
for (int i = 1; i <= 32; i++) {
        checkhip(hipMemset(d_a, 0, n * sizeof(T)));
        checkhip(hipEventRecord(startEvent, 0));
        stride<<<n / blockSize, blockSize >>> (d_a, i);
        checkhip(hipEventRecord(stopEvent, 0));
        checkhip(hipEventSynchronize(stopEvent));

        checkhip(hipEventElapsedTime(&ms, startEvent, stopEvent));
        printf("%d, %f\n", i, 2 * nMB / ms);
```

```
    }
    checkhip(hipEventDestroy(startEvent));
    checkhip(hipEventDestroy(stopEvent));
    hipFree(d_a);
}
int main(int argc, char **argv) {
    int nMB = 128;
    int deviceId = 0;
    bool bFp64 = false;
    for (int i = 1; i < argc; i++) {
        if (!strncmp(argv[i], "dev=", 4)) {
            deviceId = atoi((char *) (&argv[i][1]));
        } else if (!strcmp(argv[i], "fp64")) {
            bFp64 = true;
        }
    }
    hipDeviceProp_t prop;
    checkhip(hipSetDevice(deviceId));
    checkhip(hipGetDeviceProperties(∝, deviceId));
    printf("Device: %s\n", prop.name);
    printf("Transfer size (MB): %d\n", nMB);
    printf("%s Precision\n", bFp64 ? "Double" : "Single");
    if (bFp64) {
        runTest<double>(deviceId, nMB);
    }
    else {
        runTest<float>(deviceId, nMB);
    }
}
```

上述程序是对 hip 的浮点数计算能力进行测试，通过对核函数 offset 以及 stride 进行计时，所得时间与数组 a 的大小相比，可以得出每秒处理的浮点型数据大小，由于核函数中基本上没有计算量，耗时都是来自于全局内存中的数据读取，故而可以推出在核函数中全局内存的读写带宽。

对上面的程序进行编译 hipcc hiptest.cpp -o hiptest，程序默认测试的是单精浮点数据带宽，如果加参数 fp64，测试的是双精浮点数据带宽，如下命令。

```
./hiptest          //测试单精度浮点数据带宽
./hiptest fp64     //测试双精度浮点数据带宽
```

如上，分别对单精和双精进行测试，程序运行后会输出两段结果分别为 stride 带宽和 offset 带宽，每一段都有 32 行(32 来自循环，在循环中不断对 s 加 1 传入核函数中)，每一行都有一个带宽，分别将两段的数据绘制成折线图如下所示，可以发现一些规律，对于 stride 来说其带宽是剧烈递减的，而 offset 的带宽上下微小波动。这种带宽骤减的情况会造成 DCU 程序无法高效的运行，那么这种现象是如何产生的呢。

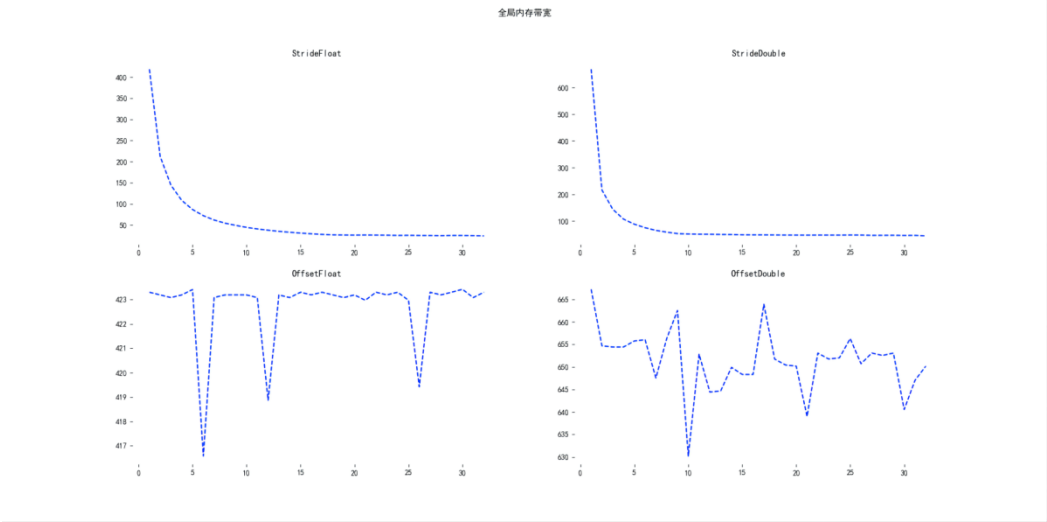


图 3.1 合并访问

在上面的测试程序中，这些现象的产生原因仅仅是不断对核函数 offset 和 stride 中的变量 s 进行加 1 操作，为什么就会造成带宽降低？或者更严谨的提出一个问题，什么情况下对全局内存读写带宽影响很大，什么情况下影响不大？要解释这一个现象，首先要介绍三种全局内存的访问模式。

1. 对齐访问

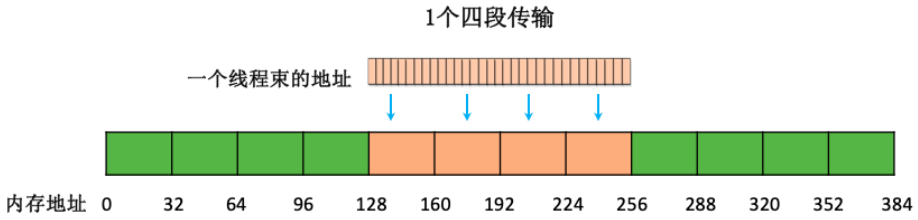


图 3.2 对齐访问

对齐访问就是线程索引与待读取的数据地址是垂直对应的，如 0 号线程读取 0 号位置的数据，对应代码中就是 i 与 blockDim.x * blockIdx.x + threadIdx.x 相等，即下面的代码中 s 等于 0。

```
template<typename T>
__global__ void offset(T *a, int s) {
    int i = blockDim.x * blockIdx.x + threadIdx.x + s;
    a[i] = a[i] + 1;
}
```

2. 偏移访问

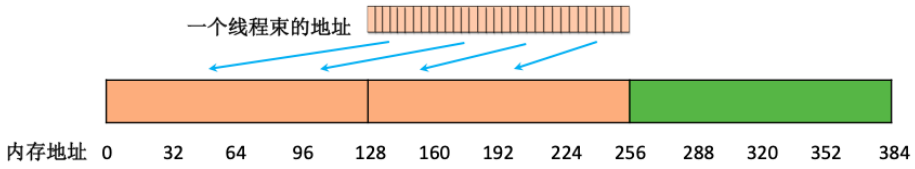


图 3.3 偏移访问

偏移访问与对齐访问类似，只是线程编号和数据位置不再垂直对应，而是统一的向前或者后面偏移一个固定量 s，s 不为 0。对齐访问和偏移访问都有一个特点就是相邻的两个线程访问的也是连续的地址空间。

3. 跨距访问

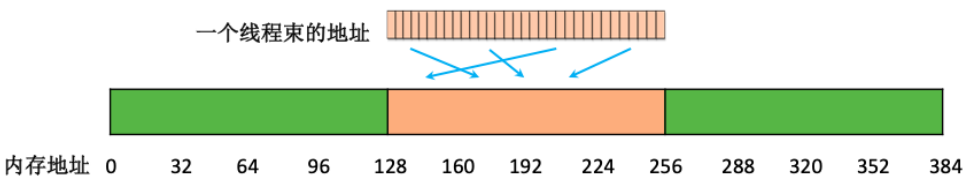


图 3.4 乱序跨距访问

跨距访问指的是线程和数据地址之间没有一个固定的关系或者是相邻线程访问的数据地址空间不是连续的而是跳跃的。

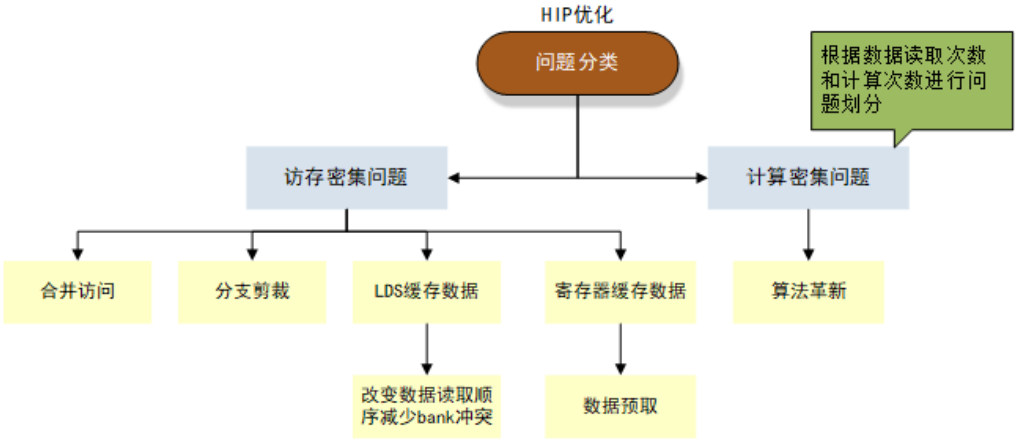
```
template<typename T>
__global__ void stride(T *a, int s) {
    int i = (blockDim.x * blockIdx.x + threadIdx.x) * s;
    a[i] = a[i] + 1;
}
```

在 stride 核函数中线程编号乘以一个常量 s 才是所要读取的数据编号，在测试程序中 s 不断被累加，随着 s 的增大，相邻线程访问的数据地址距离也越来越远。举例来说 1 号线程和 2 号线程，当 s 等于 2 的时候分别访问 a[2]和 a[4]，而当 s 等于 16 的时候分别访问 a[16]，a[32]。随着 s 的不断增大，核函数耗时越来越长，这样就造成了带宽的急剧下降。

- 结论：

通过上面的实验可以得出对全局内存进行读写时，应该尽量的让连续的线程读取连续的地址空间，这样可以提高全局内存读取的带宽。在优化核函数的时候，如何提高全局内存数据读取效率是一个关键点，因为数据没有加载完毕时 ALU 是空闲状态，想象一下 DCU 有一半的 ALU 都处在等待数据中是多么浪费性能的一件事。

4 总结



保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区